

Jour 2 — Matin : Chapitre 3 — RAG et écosystème MCP

Niveau : DÉBUTANT — Durée : ~3h30 (matinée du jour 2)

Bonjour ! Récap du jour 1

Hier, vous avez :

- Compris ce qu'est un LLM, comment il apprend, comment on le facture.
- Installé un assistant IA dans votre IDE.
- Appris à écrire des prompts structurés (canevas ROCDC-IO).

Aujourd'hui, on s'attaque au **vrai problème** que vous allez rencontrer en entreprise :

Mon LLM ne connaît pas mes documents, mon code privé, ma base de données.
Comment je fais ?

Deux réponses : **RAG** et **MCP**. On va voir les deux ce matin.

Objectifs du matin

1. Comprendre **pourquoi** un LLM seul ne suffit pas en entreprise.
2. Connaître le pipeline **RAG** (Retrieval-Augmented Generation) — étape par étape.
3. Coder un **mini-RAG** en moins de 50 lignes Python.
4. Connaître les **limites** du RAG.
5. Comprendre **MCP** (Model Context Protocol) — le standard 2024.
6. Connecter votre IDE à **2 serveurs MCP** existants.

Plan de la matinée

Heure	Section	Durée
09:00	1. Glossaire & problème à résoudre	15 min
09:15	2. Pipeline RAG expliqué avec une analogie	30 min
09:45	3. Mini-RAG en Python (TP guidé)	45 min
10:30	Pause	15 min
10:45	4. Limites du RAG en entreprise	20 min
11:05	5. MCP : le protocole standard	30 min

Heure	Section	Durée
11:35	6. Marketplace MCP — serveurs prêts à l'emploi	15 min
11:50	TP 3 — RAG sans framework + MCP marketplace	40 min
12:30	Pause déjeuner	

1. Glossaire et problème à résoudre

Mot	Définition simple
RAG	Retrieval-Augmented Generation. On récupère des morceaux pertinents avant de générer .
Embedding	Vecteur de chiffres qui « capture » le sens d'un texte.
Vecteur	Une liste de chiffres (par ex. 384 ou 1536 chiffres).
Base vectorielle	Une base de données spécialisée pour stocker et chercher des vecteurs.
Similarité cosinus	Une mesure entre 0 et 1 qui dit si 2 vecteurs (= 2 textes) sont proches sémantiquement.
Chunk	Un morceau de document (200-1000 tokens). On chunk avant d'embedder.
Top-k	Les k résultats les plus pertinents.
MCP	Model Context Protocol. Un protocole pour donner à l'IA des outils et des ressources .
Tool	Une fonction que le LLM peut appeler (ex : lire un fichier, query SQL).
Resource	Une donnée que le LLM peut lire (ex : un fichier, une page wiki).

Le problème en entreprise

Un LLM **ne connaît pas** :

- vos repos privés,
- votre wiki interne (Confluence, Notion),
- vos tickets Jira/Linear,
- votre base de données de production,
- la documentation **après sa date d'entraînement**.

3 solutions possibles :

Solution	Avantages	Inconvénients
Fine-tuning	Modèle « imprégné » de vos données	Cher, lent, opaque, vite obsolète
RAG	Données toujours à jour, droit à l'oubli simple	Qualité du retrieval critique
MCP	Accès live aux systèmes	Plus complexe à mettre en place

💡 RAG et MCP sont **complémentaires**, pas concurrents.

2. Le pipeline RAG, expliqué avec une analogie

Analogie : le bibliothécaire intelligent

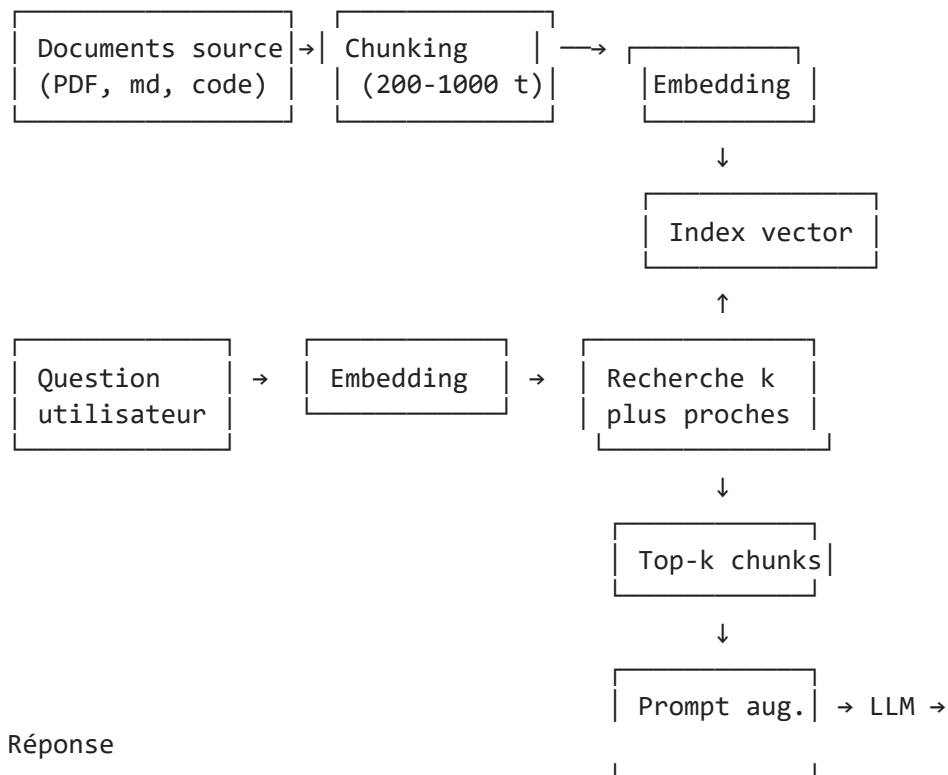
Imaginez un **bibliothécaire** dans une bibliothèque géante :

1. **Vous arrivez** avec une question : « *J'ai besoin d'un livre sur la cuisine vegan.* »
2. Le bibliothécaire **comprend votre intention** (cuisine + vegan).
3. Il **va chercher** dans les rayons 3 livres pertinents.
4. Il vous les **présente** ouverts aux pages utiles.
5. **Vous** lisez les pages et formulez la réponse.

Dans le RAG, c'est **pareil** :

1. **Vous** posez une question à l'IA.
2. Un **embedding** transforme votre question en vecteur.
3. Une **base vectorielle** trouve les 3 morceaux de documents les plus proches.
4. Ces 3 morceaux sont **insérés** dans le prompt envoyé au LLM.
5. Le **LLM** lit ces extraits et formule la réponse.

Le schéma complet



Les 6 étapes en mots

1. **Chunking** : on découpe chaque document en morceaux de 200 à 1 000 tokens (avec un peu de recouvrement entre morceaux).
2. **Embedding** : chaque morceau est transformé en un vecteur (par ex. 384 chiffres).
3. **Indexation** : ces vecteurs sont stockés dans une base spécialisée (FAISS, Chroma, pgvector...).
4. **Recherche** : la question est embeddée, on cherche les `k` vecteurs les plus proches.
5. **Augmentation** : on insère ces morceaux dans le prompt envoyé au LLM, avec la consigne « *utilise UNIQUEMENT ces extraits* ».
6. **Génération** : le LLM produit la réponse, en citant les sources.

3. Mini-RAG en Python — TP guidé

On va coder un RAG **complet** en moins de 50 lignes. Vous allez voir : ce n'est **pas** sorcier.

Installation des dépendances

`pip install sentence-transformers numpy`

`sentence-transformers` est une lib gratuite qui télécharge et utilise des modèles d'embedding open source.

```
In [ ]: # =====
# Mini-RAG complet, en 5 étapes très commentées
# =====

# --- Etape 0 : importer les librairies
import numpy as np # tableaux de chiffres
from sentence_transformers import SentenceTransformer # modele d'embedding

# --- Etape 1 : préparer un petit corpus de documents
# (dans la vraie vie, on charge depuis des fichiers .md, .pdf, etc.)
documents = [
    "Python a été créé par Guido van Rossum en 1991.",
    "FastAPI est un framework web asynchrone pour Python.",
    "Pytest est le framework de test le plus populaire en Python.",
    "MCP est un protocole publié par Anthropic en 2024.",
    "Un embedding est un vecteur dense qui capture le sens d'un texte.",
    "Claude est un assistant IA développé par Anthropic.",
    "Le langage Go a été créé par Google en 2007.",
]
print(f"Corpus chargé : {len(documents)} documents")

# --- Etape 2 : charger un modèle d'embedding
# 'all-MiniLM-L6-v2' est petit (90 Mo), rapide, gratuit, et marche bien
# Première exécution : téléchargement automatique (~90 Mo)
modele_embedding = SentenceTransformer('all-MiniLM-L6-v2')
print("Modèle d'embedding chargé.")
```

```

# --- Etape 3 : embedder TOUS Les documents
# encode() transforme une liste de textes en un tableau (n_docs, dim_embedding)
# Ici dim = 384, donc on obtient un tableau de forme (7, 384)
vecteurs = modele_embedding.encode(documents)
print(f"Vecteurs : forme {vecteurs.shape}") # (7, 384)

# --- Etape 4 : fonction de recherche
def chercher(question, k=2):
    """Renvoie les k documents les plus proches semantiquement."""
    # 4a. Embedder la question
    vec_question = modele_embedding.encode([question])[0] # forme (384,)

    # 4b. Calculer la similarite cosinus avec chaque document
    # cos(A, B) = (A.B) / (|A| * |B|)
    produit_scalaire = vecteurs @ vec_question
    norme_vecteurs = np.linalg.norm(vecteurs, axis=1)
    norme_question = np.linalg.norm(vec_question)
    similarites = produit_scalaire / (norme_vecteurs * norme_question)

    # 4c. Trier par similarite decroissante et prendre les k premiers
    # argsort() trie en ordre croissant, [::-1] retourne, [:k] coupe
    indices_top = similarites.argsort()[::-1][:k]

    # 4d. Retourner les documents et leurs scores
    return [(documents[i], float(similarites[i])) for i in indices_top]

# --- Etape 5 : tester
question = "Qui a cree Python ?"
print(f"\nQuestion : {question}")
for doc, score in chercher(question, k=2):
    print(f" Score {score:.3f} | {doc}")

```

Et maintenant, l'augmentation + génération

On vient de récupérer les `k` documents pertinents. Étape suivante : les **insérer dans un prompt** envoyé à un LLM.

```

In [ ]: # =====
# Etape 6 : prompt augmenté + appel LLM (ici simulé)
# =====

# Template du prompt augmenté
TEMPLATE = """Tu reponds UNIQUEMENT en t'appuyant sur le contexte ci-dessous.
Si la reponse n'est pas dans le contexte, dis : "Je ne sais pas".
Cite tes sources entre crochets, par ex. [doc 2].

CONTEXTE :
{contexte}

QUESTION : {question}

REPONSE :"""

def repondre(question, k=2):

```

```

"""RAG complet : recherche + prompt augmente + LLM"""
# 1. Recherche des k chunks Les plus pertinents
chunks = chercher(question, k=k)

# 2. Construire Le contexte (numérote pour citation)
contexte = "\n".join(
    f"[doc {i+1}] {doc}"
    for i, (doc, _score) in enumerate(chunks)
)

# 3. Construire Le prompt complet
prompt = TEMPLATE.format(contexte=contexte, question=question)

# 4. Appeler Le LLM (ici simule)
# Dans la vraie vie : ollama.generate() ou openai.ChatCompletion.create()
# Pour la demo, on affiche juste Le prompt pour qu'on Le voie
print("=" * 60)
print("PROMPT QUI SERAIT ENVOYE AU LLM :")
print("=" * 60)
print(prompt)
print("=" * 60)
return prompt

# --- On essaie
repondre("Qui a cree Python ?", k=3)

```

Ce qu'on vient de construire

Vous avez maintenant un **RAG complet** :

- Indexation de documents → embedding → stockage vectoriel
- Recherche par similarité
- Augmentation du prompt
- (Génération à brancher sur Ollama ou une API)

C'est exactement ce que font les outils comme LangChain, LlamaIndex, Haystack, etc. — en plus simple et plus contrôlable.

✅ **Checkpoint** : sauriez-vous dessiner le pipeline RAG sans regarder ? Si oui → continuez. Si non → faites le schéma sur papier.

4. Limites du RAG en entreprise

Le RAG n'est pas magique. Voici les **6 problèmes classiques** et leurs solutions.

Limite	Cause	Atténuation
Réponses hors contexte	Le LLM ignore la consigne « <i>utilise uniquement le contexte</i> »	Filtrer la sortie, ajouter un fallback « <i>je ne sais pas</i> »

Limite	Cause	Atténuation
Mauvais chunks récupérés	Chunking mal dimensionné, embedding faible	Re-ranking, hybrid search (BM25 + vectoriel)
Coût élevé	k trop grand → context window saturé	Réduire k, compresser sémantiquement
Données obsolètes	Index pas rafraîchi	Pipeline d'ingestion incrémentale
Sources non citées	Modèle paraphrase sans citer	Format <code>[doc_id]</code> imposé dans la sortie
Sécurité	Fuite de docs sensibles	Filtrer par ACL avant le top-k

Quand le RAG est-il une bonne idée ?

- ✓ Corpus **relativement stable** (doc projet, wiki, livres).
- ✓ Volume **gérable** (de 100 à quelques millions de chunks).
- ✓ Besoin de **citations** vérifiables.
- ✗ Données **temps réel** (préférer MCP).
- ✗ Logique métier complexe (préférer des fonctions ou MCP).

5. MCP — Model Context Protocol

Le problème que MCP résout

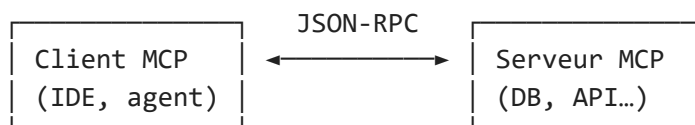
Avant MCP (fin 2024), chaque outil IA avait sa **propre façon** de connecter des outils externes :

- ChatGPT plugins
- OpenAI function calling
- Claude tools
- Cursor commands
- etc.

→ **Chaque intégration était à refaire pour chaque outil.**

MCP standardise ça. **Un serveur MCP = compatible avec tous les clients MCP.**

L'architecture en 3 mots



- **Client** : Claude Desktop, Claude Code, Cursor, Continue, votre app custom.
- **Serveur** : un processus que vous lancez, qui expose des outils.
- **Transport** : stdio (local, le + courant) ou HTTP/SSE (réseau).

Ce qu'un serveur MCP peut exposer

Type	Description	Exemples
Tools	Fonctions appelables par le LLM	<code>read_file</code> , <code>query_sql</code> , <code>send_email</code>
Resources	Données lisibles à une URI	<code>db://users/schema</code> , <code>file:///path/doc.md</code>
Prompts	Templates partagés	<code>pr-review-template</code>

Pourquoi c'est utile ?

- 🔌 **Plug-and-play** : 1 serveur = compatible N clients.
- 🧱 **Composable** : empiler plusieurs serveurs (filesystem + GitHub + Postgres).
- 🛡️ **Sécurité** : pas de clé API dans le prompt — le **client** tient les tokens.
- 🌐 **Ouvert** : spec publique, déjà 100+ serveurs sur la marketplace.

6. Marketplace MCP — les serveurs prêts à l'emploi

Voici les **10 serveurs MCP les plus utiles** pour un dev :

Serveur	Usage	Quand l'utiliser
filesystem	Lire/écrire des fichiers locaux	Quasi tout le temps
github	Issues, PR, commits, recherche de code	Tous les jours en dev
gitlab	Idem pour GitLab	Selon votre infra
postgres / sqlite	Requêter une base	Explorer un schéma
slack	Lire/écrire dans des canaux	Notifications, recherche
memory	Mémoire long terme entre sessions	Assistant personnel
brave-search	Recherche web	Compenser le cut-off
puppeteer / playwright	Automatiser un navigateur	Web scraping, tests E2E
time	Date et heure courantes	Corriger les hallucinations temporelles
sentry	Lire les erreurs de prod	Debug rapide

Catalogue à jour : <https://github.com/modelcontextprotocol/servers>

Comment installer un serveur MCP ?

La plupart des serveurs sont des paquets npm. On les déclare dans le fichier de config du client (par exemple Claude Desktop).

```
In [ ]: # =====
# Exemple : configurer Claude Desktop avec 2 serveurs MCP
# =====
# Fichier à éditer : ~/Library/Application Support/Claude/claude_desktop_config.json
# (macOS) ou %APPDATA%/Claude/claude_desktop_config.json (Windows)

import json

config = {
    "mcpServers": {
        # --- Serveur 1 : accès filesystem (sandbox dans un dossier spécifique)
        "filesystem": {
            "command": "npx",
            "args": [
                "-y",
                "@modelcontextprotocol/server-filesystem",
                "/Users/moi/projets"
            ]
        },
        # --- Serveur 2 : GitHub
        "github": {
            "command": "npx",
            "args": ["-y", "@modelcontextprotocol/server-github"],
            "env": {
                # Le client passe la clé en variable d'env (le LLM ne la voit pas)
                "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_XXXXXXXXXXXX"
            }
        }
    }
}

# Afficher le JSON formaté
print(json.dumps(config, indent=2))

# Étapes :
# 1. Créer un Personal Access Token sur github.com/settings/tokens (scopes: repo, r
# 2. Coller la config dans claude_desktop_config.json
# 3. Redémarrer Claude Desktop
# 4. Vérifier l'icône "outils" en bas de l'UI -> doit montrer 2 serveurs connectés
```

TP 3 — RAG sans framework + MCP marketplace

Durée : 40 min — Modalité : binôme

Partie A — Améliorer le mini-RAG (20 min)

À partir du code de la section 3, ajoutez :

1. **Chargement de 5 fichiers** `.md` dans un dossier `./docs/` (votre doc projet, wiki, README de vos projets, etc.).
2. **Chunking** par **200 mots** avec **40 mots de recouvrement**.
3. **Évaluation** sur 5 questions de votre choix : bonne réponse ? source citée ?

Indice : pour lister vos fichiers `.md`, utilisez :

```
from pathlib import Path
fichiers = list(Path("./docs").rglob("*.md"))
```

Partie B — MCP marketplace (20 min)

1. Installer **Claude Desktop** OU **Continue** dans VS Code.
2. Configurer **2 serveurs MCP** : `filesystem` (dossier de votre projet) + `github` (token PAT).
3. Dans une session, demander à l'IA :
 - « *Liste les fichiers TODO dans le projet* » → via filesystem.
 - « *Crée une PR avec ces changements* » → via github.
4. Observer la **trace** des appels (icône outils dans l'UI).

Livrables

- Code Python amélioré du mini-RAG.
- Captures d'écran des 2 démos MCP.
- Fichier `tp3_observations.md` : qu'est-ce qui a marché ? Qu'est-ce qui a échoué ?

Corrigés : `corriges/jour2/CORRIGE_chapitre3.ipynb`.

Synthèse de la matinée

Concept	À retenir en une phrase
Problème entreprise	Le LLM ne connaît ni vos données, ni vos systèmes.
RAG en 6 étapes	Chunk → Embed → Index → Search → Augment → Generate.
Embedding	Vecteur de chiffres qui capture le sens.
Similarité cosinus	Mesure entre 0 et 1 — proche de 1 = textes proches.
Limites RAG	Réponses hors contexte, chunks ratés, coût, sécurité.
MCP	Protocole standard pour exposer des outils à un LLM.
Marketplace MCP	100+ serveurs prêts à l'emploi (filesystem, github...).

Concept**À retenir en une phrase**

Combinaison gagnante RAG pour la doc stable + MCP pour les systèmes vivants.

🍽️ Bon appétit ! Cet après-midi, on **crée notre propre serveur MCP** et on découvre les **agents autonomes**.